

Kerberos/NFS

역할: FARM/LAB에서 AD Kerberos로 사용자를 인증하고, 같은 UID/GID로 NFS 공유 스토리지에 접근하게 하는 구조와 운영 기준을 설명한다.

이 문서는 Kerberos/NFS 문서의 시작점이다. 처음 읽는 사람은 **설계**에서 전체 인증 흐름을 먼저 이해한 뒤 **운영**에서 생성·점검·복구 명령을 확인한다. 과거 장애의 증거와 재현 실험은 **디버깅 로그**에서 확인한다. 환경별 실제 값과 최종 실행 절차는 인프라 저장소의 FARM/LAB canonical runbook에서 확인한다.

문서 구성

문서	읽어야 하는 경우	핵심 내용
현재 페이지	전체 범위와 문서 위치를 빠르게 확인할 때	목적, 원칙, 관리 범위
설계	인증이 어느 서버와 프로세스를 거치는지 이해할 때	FARM/LAB NFS version·principal 차이, keytab 발급, RPCSEC_GSS, 권한 판정
운영	계정/컨테이너 생성, timer 확인, mount·KVNO 장애를 처리할 때	Synology/Linux storage 차이, 명령, 점검표, 수동 repair/rotation
디버깅 로그	재현된 장애의 원인과 실험 조건·명령·증거를 확인할 때	장애별 증상, 가설, 재현, 판정, 복구·예방

한눈에 보는 구조

관리 서버(uidctl)

- AD DC: 사용자·RFC2307·keytab 생성
- 계산 호스트: root-only keytab + ccache refresh timer
- NAS/storage: NFS service keytab + 사용자 home
- container: keytab이 아닌 ccache만 사용

사용자 I/O

container process -> host kernel NFS client -> rpc.gssd

-> AD KDC ticket -> NAS svcgssd/NFS -> UID/GID 권한 판정

현재 구성에서 지켜야 할 원칙은 다음과 같다.

- 사용자 keytab은 장기 자격증명이므로 계산 호스트의 `/etc/decs-krb/keytabs/<username>.keytab`에 `root:root0400`으로만 둔다.

- 컨테이너에는 keytab을 넣지 않는다. 호스트가 만든 `/run/user/<uid>/krb5cc` 와 읽기 전용 `/etc/krb5.conf` 만 bind mount한다.
- 컨테이너가 시작되기 전에 사용자별 `decs-krb-refresh@<username>.timer` 를 활성화하고 유효한 ccache를 한 번 발급한다.
- NFS mount source는 IP가 아니라 NFS service principal과 일치하는 FQDN을 쓴다. FARM의 source는 `nas.farm.decs.internal:/volume1/share` 다.
- `addr=100.100.100.120` 은 FQDN이 해석된 실제 전송 주소로 사용할 수 있다. source 자체를 `100.100.100.120:/...` 로 바꾸면 안 된다.
- 기본 security flavor는 `sec=krb5` 다. `krb5i` 와 `krb5p` 는 무결성·암호화가 명시적으로 필요한 경로에서 성능 비용과 함께 선택한다.

Identity 불변 조건

한 사용자에게 대해 다음 숫자가 같아야 한다.

```
UID DB ubuntu_uid
= AD RFC2307 uidNumber
= NFS client에서 관측한 home owner UID
= container UID
```

primary GID도 같은 원칙 적용

이 조건을 확인하지 않고 NAS에서 `chown` 만 반복하면 AD, DB, NAS의 identity가 더 어긋날 수 있다. 계정 생성 흐름은 Docker와 DB를 확정하기 전에 AD/NAS/host identity와 실제 Kerberos NFS 쓰기를 검증한다.

모듈별 관리 범위

작업	담당 모듈
AD 사용자·그룹, 사용자 keytab, host ccache timer, container 생성	<code>user-lifecycle</code>
공통 host Kerberos/NFS desired state와 fstab	<code>server-state</code>
NAS NFS service account·SPN·service keytab의 수동 변경	<code>kerberos-nfs</code>
GSS readiness, keytab drift 관측, mount 상태, canary, forensics	<code>monitoring</code>
ccache 소비, 시작 전 credential 확인, restricted sudo	<code>container-images</code>

관측 실패가 곧바로 AD password 변경이나 NAS service 재시작으로 이어지지 않게 읽기 전용 monitoring과 상태 변경 작업을 분리한다.

기준 문서와 코드

- [FARM canonical runbook](#)
- [LAB canonical runbook](#)
- [user-lifecycle Kerberos 구현](#)
- [container 생성 구현](#)
- [host Kerberos/NFS desired state](#)
- [Kerberos/NFS keytab health check](#)
- [NFS GSS monitoring](#)

실제 endpoint, principal, mount option이 바뀌면 이 위키보다 canonical runbook과 코드 설정을 먼저 갱신하고, 검증 시각과 대상 host를 함께 기록한다.

Kerberos/NFS 설계

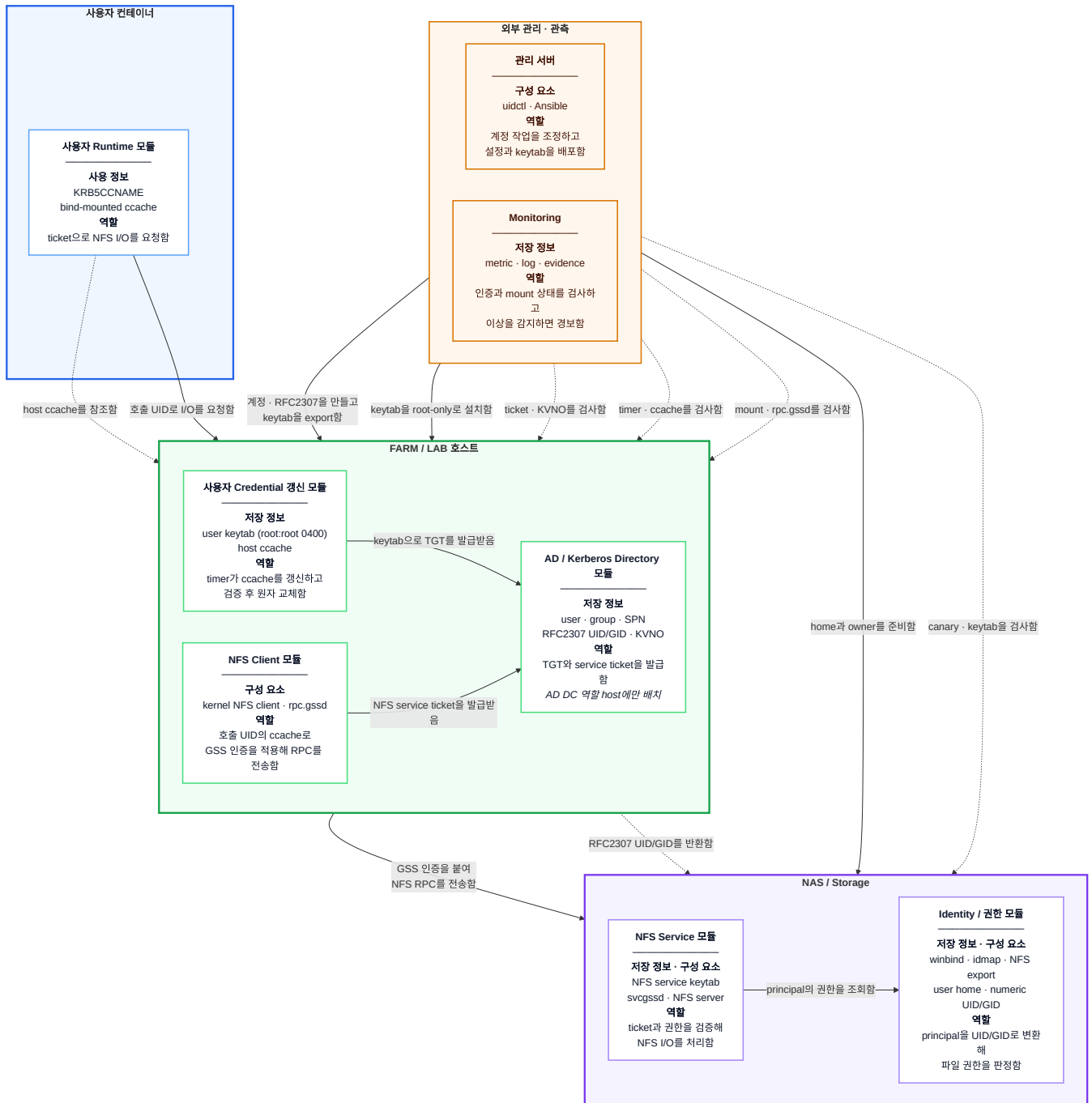
이 문서는 keytab이 만들어지는 시점부터 사용자가 NFS 파일을 읽고 쓰는 시점까지, 어느 서버의 어떤 객체와 프로세스가 관여하는지를 설명한다.

1. 설계 목표

- 비밀번호를 컨테이너에 저장하지 않고 사용자별 Kerberos 인증을 제공한다.
- DB, AD, 계산 호스트, NAS와 컨테이너가 같은 numeric UID/GID를 사용한다.
- NFS 서버는 FQDN 기반 service principal로 인증하고 client는 `sec=krb5` 로 RPCSEC_GSS context를 만든다.
- keytab 같은 장기 자격증명의 노출 범위를 host root로 제한한다.
- ticket 갱신과 컨테이너 lifecycle을 분리하여 컨테이너 재시작 중에도 credential을 안정적으로 유지한다.
- monitoring은 drift를 관측하되 공유 NAS와 AD를 자동으로 변경하지 않는다.

2. 전체 구성

다이어그램의 **굵은 바깥 제목은 배치 영역**이고, 영역 안 각 카드의 **굵은 첫 줄은 모듈 이름**이다. 카드 안의 **저장 정보 / 구성 요소**에는 명사만 적고, **역할**에는 **주체가 무엇을 하는지 동사로** 적었다. 화살표에는 출발 모듈이 요청하거나 제공하는 동작을 표시했으며, 점선은 credential 참조나 읽기 전용 조회·관측을 나타낸다. AD/KDC는 모든 계산 host에 있는 것이 아니라 FARM/LAB 중 **AD DC 역할을 맡은 host에만** 있다. 다이어그램에는 핵심 값과 동작만 표시하고 상세 책임은 아래 표에서 설명한다.



구성요소별 책임

영역	모듈	내부 구성 요소	역할
외부 관리	계정 생성 조정	uidctl, Ansible runner	계정 생성 절차를 조정하고 AD, host, storage에 필요한 상태를 준비
FARM/LAB 호스트	AD/Kerberos Directory	Samba AD DC, KDC, 사용자·그룹·SPN·RFC2307 객체	principal과 Unix identity 저장, keytab export, TGT/service ticket 발급; AD DC 역할 host에만 배치
FARM/LAB 호스트	사용자 Credential 갱신	user keytab, decs-krb-refresh@.service/.timer, host ccache	root-only keytab으로 ccache를 만들고 검증 후 원자 교체

영역	모듈	내부 구성 요소	역할
FARM/LAB 호스트	NFS Client	kernel NFS client, <code>rpc.gssd</code>	호출 UID의 ccache로 RPCSEC_GSS context를 만들고 NFS RPC 전송
사용자 컨 테이너	사용자 Runtime	사용자 process, bind-mounted ccache, <code>KRB5CCNAME</code>	keytab 없이 host가 갱신한 ticket을 사용하여 NFS 파일 I/O 수행
NAS/Stora ge	NFS Service	service keytab, <code>svcgssd</code> , NFS server	<code>nfs/<fqdn>@<realm></code> service ticket을 수락하고 NFS 요청 처리
NAS/Stora ge	Identity/권한	Samba, winbind, idmap, NFS export와 filesystem	Kerberos principal을 RFC2307 UID/GID로 해석하여 최 종 파일 권한 판정
외부 관측	Monitoring	readiness, KVNO checker, mount probe, canary, forensics	AD/host/storage 상태를 읽기 전용으로 수집하고 drift와 장애를 경보

3. FARM과 LAB의 현재 설정 차이

FARM과 LAB은 “사용자 keytab은 host root만 보관하고 container에는 ccache만 전달한다”는 credential 모델은 같다. 그러나 realm, NFS server 구현, mount source, service principal과 NFS protocol version은 서로 다르다. 한 환경의 값을 다른 환경에 그대로 복사하면 KDC가 잘못된 service ticket을 발급하거나 NFS server가 ticket을 복호화하지 못한다.

3.1 Identity와 endpoint

항목	FARM	LAB
Kerberos realm	<code>FARM.DECS.INTERNAL</code>	<code>LAB.DECS.INTERNAL</code>
DNS domain / NetBIOS	<code>farm.decs.internal</code> / <code>FARM</code>	<code>lab.decs.internal</code> / <code>LAB</code>
AD/KDC	<code>dc1.farm.decs.internal</code> ; farm2가 primary이고 farm6·farm7을 replica 경로로 사용	<code>dc1.lab.decs.internal</code> ; lab2
NFS server 구현	Synology NAS가 FARM AD domain member로 동작	일반 Linux storage host가 LAB AD domain member와 NFS server로 동작
관리 SSH	<code>192.168.2.30:6954</code>	<code>192.168.1.20:6953</code>
NFS data 주소	<code>100.100.100.120</code>	<code>100.100.100.100</code>
운영 mount source	<code>nas.farm.decs.internal:/volume1/share</code>	<code>lab- storage.lab.decs.internal:/294t/dcloud/share</code>
계산 host mount target	<code>/home/tako<번호>/share</code>	<code>/home/tako<번호>/share</code>

항목	FARM	LAB
사용자 home 원본	/volume1/share/user-share/<username>	/294t/dcloud/share/user-share/<username>

관리 주소는 SSH와 장비 관리에만 사용한다. NFS source에는 반드시 service principal의 hostname을 사용한다. LAB의 /294t/dcloud/share/test_krb 와 /294t/health/nfs-gss-canary 는 각각 격리 PoC와 canary용 export이며 운영 사용자 mount source인 /294t/dcloud/share 와는 용도가 다르다.

3.2 NFS와 Kerberos service identity

항목	FARM	LAB
현재 NFS version	vers=4.0	vers=4.1
기본 security flavor	sec=krb5	sec=krb5
server export flavor	production storage IP에는 sec=krb5:krb5i:krb5p ; client가 krb5 를 선택	현재 LAB baseline은 sec=krb5
NFS service principal	nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL	nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL
추가 principal	nfs/NAS@FARM.DECS.INTERNAL , nfs/nas.ai lab.dgu@AILAB.DGU 를 NAS keytab에서 함께 보존	LAB profile에는 추가 acceptor 설정 없음
NFS SPN 등록 계정	전용 AD user svc-nfs-farm ; Synology machine account NAS\$ 와 분리	Linux storage computer account LAB-STORAGE\$
NFS service keytab	/etc/nfs/krb5.keytab 이 ticket 검증 기준이며 /etc/krb5.keytab 도 checker가 비교	/etc/krb5.keytab
server GSS process	Synology의 /usr/sbin/svcgssd ; 여러 realm의 nfs/* 를 받아야 하므로 -p 로 하나를 고정하지 않음	Linux rpc-svcgssd 또는 NFS server가 관리하는 rpc.svcgssd
client machine principal 예	FARM8\$@FARM.DECS.INTERNAL	LAB9\$@LAB.DECS.INTERNAL
GSS canary	전용 export가 없어 현재 비활성	lab-storage.lab.decs.internal:/294t/health/nfs-gss-canary , vers=4.1,sec=krb5 로 활성화

사용자 principal도 realm에 따라 <username>@FARM.DECS.INTERNAL 또는 <username>@LAB.DECS.INTERNAL 이 된다. 사용자 keytab, host machine keytab과 NFS service keytab은 서로 다른 principal과 lifecycle을 가지므로 한 파일로 합쳐서는 안 된다.

3.3 NFS version이 다른 이유

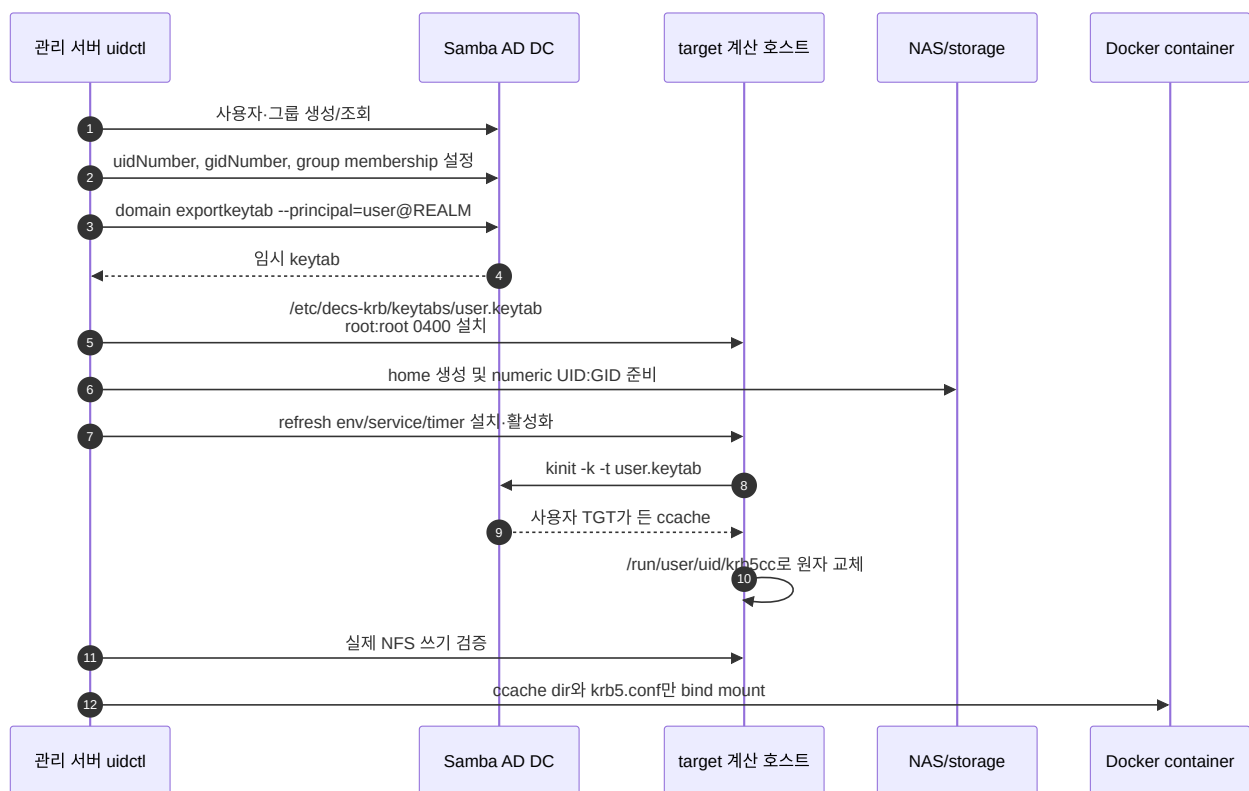
FARM의 Synology NAS에는 NFSv4.1 기능이 활성화되어 있지만, rollout에서 `vers=4.1` 과 `RPCSEC_GSS`를 함께 사용했을 때 timeout 또는 access denied가 발생했다. 따라서 현재 운영 baseline은 `vers=4.0,sec=krb5,hard,timeo=600,retrans=2` 다. `fstab`에서 1 MiB `rsz/size` 를 요청해도 실제 연결에서는 Synology와 128 KiB로 협상될 수 있다.

LAB은 Linux storage의 NFSv4.1을 사용한다. NFSv4.1 session/slot 동작은 server kernel 구현에 따라 달라지므로, 구형 kernel의 idmap deferral과 slot 고착 문제는 [NFSv4.1 session slot 고착](#)에서 별도로 관리한다. 두 환경의 version은 각각 검증된 값이므로 LAB을 임의로 v4.0으로 낮추거나 Synology NAS를 검증 없이 v4.1로 올리지는 않는다.

현재 설정값은 [server-state Kerberos/NFS playbook](#)에 정의되어 있다. Monitoring에서 기대하는 값은 `exporter` 변수에서 확인한다.

4. 사용자 keytab 발급 흐름

keytab은 컨테이너 안에서 만들지 않는다. AD DC에서 export한 뒤 target 계산 호스트의 root-only 경로에 설치한다.



실제 생성 구현은 다음 순서로 동작한다.

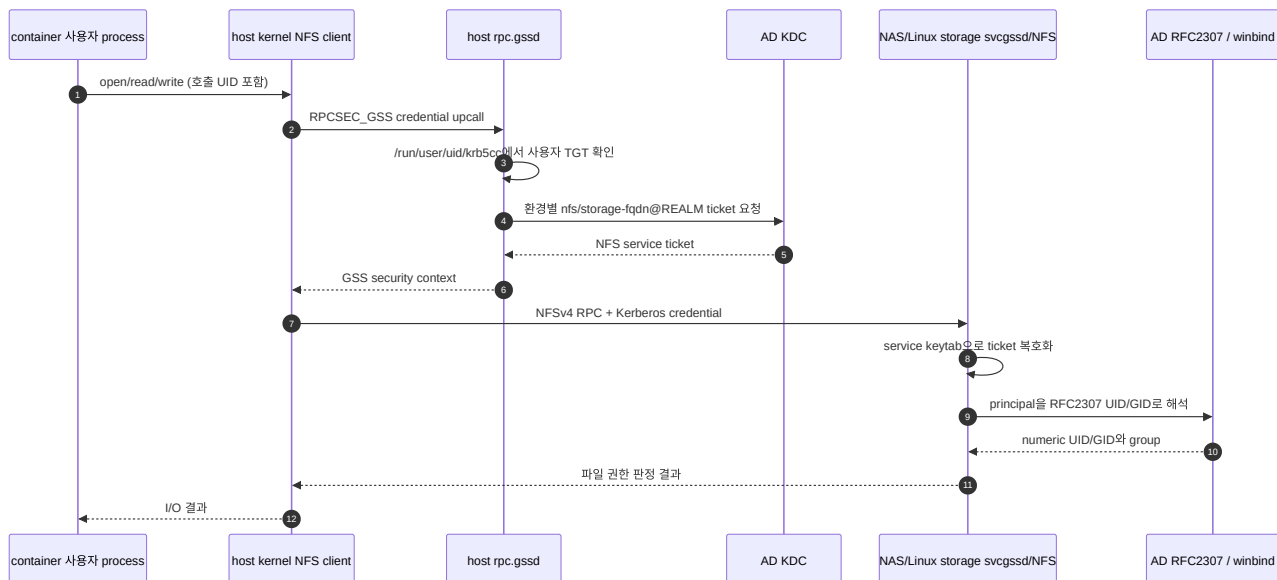
1. DB/기존 AD/storage 값을 사용해 UID/GID를 선택한다.
2. AD user/group과 RFC2307 `uidNumber`, `gidNumber` 를 보장한다.
3. AD DC에서 사용자 keytab을 임시 파일로 export하고 `0400` 으로 만든다.
4. target이 다른 host이면 Ansible `fetch` 와 `copy` 로 root-only keytab을 옮긴다.
5. NAS/storage home을 동일 UID/GID로 준비한다.
6. target host에 ccache directory, refresh helper, service/timer를 만든다.
7. host에서 사용자 ccache를 발급하고 NFS owner와 실제 write/delete를 검증한다.

8. 검증 후에만 컨테이너와 DB record를 확정한다.

관련 코드는 [AD identity](#)와 [keytab 생성](#), [container 생성 transaction](#), [credential 경로 모델](#)에서 확인한다.

5. 실제 NFS 인증 흐름

keytab 발급과 매 파일 접근의 인증은 서로 다른 흐름이다. 평상시 파일 I/O에서 컨테이너가 keytab을 직접 읽는 일은 없다.



FARM에서는 `nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL`, LAB에서는 `nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL` ticket을 요청한다. 이 값은 mount source의 hostname에서 결정되며 단순 표시용 설정이 아니다.

인증이 실패한 단계에 따라 증상이 달라진다.

실패 위치	대표 증상
사용자 keytab/ccache	<code>kinit</code> 또는 <code>klist</code> 실패, 사용자 접근만 <code>Permission denied</code>
DNS/FQDN/SPN	<code>kvno nfs/<fqdn></code> 실패, IP source mount에서 principal 불일치
host machine keytab/ <code>rpc.gssd</code>	부팅 시 mount 실패, readiness의 keytab/kinit/rpc-gssd stage 실패
NAS service keytab/KVNO	여러 client가 동시에 GSS 실패, <code>kvno -k</code> 복호화 실패
RFC2307/idmap	인증은 되지만 owner가 <code>nobody</code> 이거나 UID/GID가 달라 쓰기 거부
NFS transport/kernel	<code>not responding</code> , D-state, mount probe timeout

6. Keytab과 ccache를 분리한 이유

구분	keytab	ccache
의미	principal의 장기 비밀키	이미 발급된 TGT/service ticket 묶음
새 ticket 발급	가능	제한된 renew 기간 안에서만 가능
유출 영향	rotation할 때까지 반복 악용 가능	ticket lifetime/renew lifetime에 제한
저장 위치	host root-only	/run/user/<uid>/krb5cc, 사용자 0600
container 전달	금지	bind mount하여 전달

컨테이너 삭제만으로 유출된 keytab을 무효화할 수 없기 때문에 image, Docker volume, Kubernetes Secret에 사용자 keytab을 그대로 넣지 않는다. 호스트 refresh service는 임시 ccache에서 klist 검증을 끝낸 뒤 소유권 uid:gid, mode 0600 을 적용하고 최종 경로로 원자 교체한다.

현재 코드가 구현하는 대상은 Docker container다. Kubernetes Pod도 같은 원칙을 적용해야 하지만, Pod가 다른 node로 이동할 수 있으므로 단순 hostPath와 특정 node의 systemd timer에 의존해서는 안 된다. Pod 지원을 추가할 때는 node 배치, credential 발급 controller/CSI, rotation과 Pod 종료 시 정리를 별도 설계해야 한다.

7. Ticket 갱신 설계

decs-krb-refresh@<username>.timer 는 기본적으로 boot 2분 뒤 시작하고 사용자별 설정 주기(현재 일반적으로 1시간)마다 oneshot service를 실행한다.

유효 ccache 있음

- └─ renew deadline이 margin보다 멀 -> 임시 복사본에서 kinit -R
- └─ deadline 임박/renew 실패 -> root-only keytab으로 새 ticket 발급

발급 결과 -> klist 검증 -> uid:gid 0600 -> 최종 ccache로 atomic move

기본 재발급 여유는 DECS_KRB_REISSUE_BEFORE_SECONDS=86400 (24시간)이다. 기존 ticket의 PAC/group 정보는 renew 시 유지될 수 있으므로 AD group 변경 뒤에는 keytab을 이용한 fresh login을 한 번 강제해야 한다.

8. NFS service identity와 KVNO

FARM NFS SPN은 Synology domain member의 machine account NAS\$ 가 아니라 전용 AD service account svc-nfs-farm 에 등록되어 있다.

nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL
nfs/NAS@FARM.DECS.INTERNAL

과거에는 `NAS$` machine password가 주기적으로 변경될 때 KVNO도 바뀌어 NAS의 정적 NFS keytab과 어긋날 수 있었다. 이를 service identity와 domain membership의 lifecycle 결합 문제로 보고 NFS SPN을 전용 계정으로 분리했다.

현재 `svc-nfs-farm`에는 자동 password expiration/rotation timer가 없다. 따라서 “NAS KVNO가 지금도 주기적으로 바뀐다”가 아니라 **과거 자동 변경 문제를 전용 계정으로 제거했고, 현재 KVNO는 관리자가 명시적으로 rotation할 때만 바뀐다**가 정확한 운영 모델이다. 별도 5분 checker는 변경을 만들지 않고 AD KVNO와 두 NAS keytab의 일치만 확인한다.

NAS의 `svcgssd`는 FARM과 AILAB principal을 한 keytab에서 받아들이므로 `-p`로 FARM principal 하나만 강제하지 않는다. keytab repair 시에도 AILAB acceptor를 보존한다.

LAB은 Synology NAS용 전용 service account와 keytab merge 절차를 사용하지 않는다. 현재 LAB keytab checker profile은 `LAB-STORAGE$` computer account의 KVNO와 Linux storage의 `/etc/krb5.keytab` 안 `nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL`을 비교한다. FARM repair나 rotation script를 LAB storage에 실행하면 안 된다.

9. FQDN mount와 service principal

Kerberos의 NFS service identity는 host-based principal이다. FARM mount source는 다음처럼 principal의 hostname과 같아야 한다.

```
mount source: nas.farm.decs.internal:/volume1/share
service SPN:  nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL
transport:    addr=100.100.100.120
```

`100.100.100.120:/volume1/share`를 source로 쓰면 client가 IP 기반 service identity를 찾으려 하거나 기대한 FQDN SPN과 연결하지 못할 수 있다. 반면 source를 FQDN으로 유지한 상태에서 runtime option에 `addr=100.100.100.120`이 보이는 것은 정상이다. 관리 SSH 주소 `192.168.2.30`은 NFS transport로 사용하지 않는다.

LAB도 같은 규칙을 적용한다.

```
mount source: lab-storage.lab.decs.internal:/294t/dcloud/share
service SPN:  nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL
transport:    100.100.100.100
```

LAB 관리 SSH 주소 `192.168.1.20`이나 data IP `100.100.100.100:/294t/dcloud/share`를 mount source로 직접 쓰지 않는다.

10. 보안 flavor와 권한 모델

- `sec=krb5`: 사용자/서비스 인증. RPC payload 무결성·암호화 wrapping 없음.
- `sec=krb5i`: 인증 + RPC payload 무결성.

- `sec=krb5p`: 인증 + 무결성 + privacy encryption.

현재 내부 FARM/LAB 기본은 `sec=krb5` 다. packet capture에 payload가 포함될 수 있으므로 NFS forensics 산출물은 root-only로 보관한다.

Kerberos 인증 뒤 최종 파일 권한은 numeric UID/GID와 mode/ACL로 결정된다. 컨테이너 root가 host credential 경계를 우회하지 못하도록 Kerberos mode에서는 `DECS_USER_SUDO_MODE=restricted` 를 함께 사용한다. 다만 restricted sudo는 완전한 sandbox가 아니므로 강한 격리가 필요하면 sudo와 `SYS_ADMIN` , host bind mount 범위도 별도로 줄여야 한다.

11. 설정과 구현 위치

관심사	기준 코드/문서
FARM endpoint, SPN, mount option, NAS keytab	FARM runbook
LAB topology와 rollout gate	LAB runbook
사용자 AD/keytab/ccache 명령 생성	commands.py
keytab·ccache·home 경로	paths.py
create-container transaction과 Docker bind	create_container.py
container credential 대기와 restricted sudo	entrypoint.sh
host machine keytab/readiness/fstab	kerberos_nfs_client_recovery.yml
NFS GSS readiness와 recovery	cluster-monitor-exporter scripts
NAS service keytab drift checker	check-nfs-keytab.sh

Kerberos/NFS 운영

이 문서는 현재 Docker 기반 FARM/LAB 환경의 일상 점검과 장애 대응 절차를 설명한다. 환경별 endpoint와 적용 상태는 반드시 canonical runbook에서 다시 확인한다.

1. 운영 원칙

1. 관측과 변경을 분리한다. 5분 keytab checker는 자동 repair/rotation을 하지 않는다.
2. 사용자 keytab은 host root-only로 두고 container에는 ccache만 전달한다.
3. 컨테이너를 시작하기 전에 refresh timer와 최초 ccache 발급을 완료한다.
4. mount source는 FQDN을 사용하고 IP는 `addr=` transport 값으로만 사용한다.

5. `findmnt`, `klist`, `kvno`, readiness 순서로 원인을 좁힌 뒤 service restart나 remount를 마지막 수단으로 사용한다.
6. UID/GID 불일치는 AD·DB·NFS 숫자를 비교한 뒤 고친다. 먼저 `chown` 하지 않는다.

2. Kerberos container 생성 (추후 자동화 시스템으로 대체될 예정)

대표 CLI 형태는 다음과 같다. 실제 image/version, 사용자 정보와 대상 server는 요청에 맞게 지정한다.

```
cd /home/jy/server_manage/user-lifecycle/script

python3 -B -m uid_manager.cli create-container \
  --server-id FARM8 \
  --name "사용자 이름" \
  --username <username> \
  --group <groupname> \
  --image <image> \
  --version <version> \
  --created-by <operator> \
  --email <email> \
  --phone <phone> \
  --enable-kerberos
```

먼저 `--dry-run`으로 UID/GID, target host, mount source, container command를 확인하는 것을 권장한다. 기존 사용자 password/key를 의도적으로 바꿀 때만 `--rotate-kerberos-keytab`을 추가한다. 이 옵션은 AD user password와 KVNO를 바꾸므로 단순 재배포 옵션으로 사용하지 않는다.

생성 transaction은 다음 조건을 모두 통과한 후 Docker/DB를 확정해야 한다.

- AD `uidNumber/gidNumber`가 선택한 DB UID/GID와 일치한다.
- target host keytab이 `root:root 0400`이다.
- ccache timer가 enabled/active이고 최초 ccache가 유효하다.
- NAS/storage home의 numeric owner가 기대한 UID/GID다.
- 새로 만든 사용자 credential로 host NFS write/delete가 성공한다.
- Docker에는 ccache directory와 읽기 전용 `krb5.conf`만 전달된다.

구현은 `create_container.py`와 Kerberos command builder에서 확인한다.

3. Host keytab과 ccache 확인

파일 권한

```
sudo stat -c '%U:%G %a %n' \
    /etc/decs-krb/keytabs/<username>.keytab \
    /etc/decs-krb/refresh.d/<username>.env

sudo stat -c '%U:%G %a %n' /run/user/<uid> /run/user/<uid>/krb5cc
```

기대값은 다음과 같다.

```
keytab:      root:root 400
refresh env: root:root 600
ccache dir:  <uid>:<gid> 700
ccache:      <uid>:<gid> 600
```

timer와 ticket

```
systemctl list-timers 'decs-krb-refresh@*.timer'
systemctl status 'decs-krb-refresh@<username>.timer' --no-pager
sudo systemctl start 'decs-krb-refresh@<username>.service'
sudo journalctl -u 'decs-krb-refresh@<username>.service' -n 100 --no-pager

sudo -u '#<uid>' env KRB5CCNAME=FILE:/run/user/<uid>/krb5cc \
    klist -c FILE:/run/user/<uid>/krb5cc
```

컨테이너 생성 때 timer unit을 설치·enable/start하고, 기존 ccache가 유효하지 않으면 oneshot service를 즉시 실행하도록 구현되어 있다. timer만 존재하고 최초 ccache가 없는 상태에서 컨테이너를 먼저 시작하면 Kerberized home 접근이 실패할 수 있다.

AD group을 변경했다면 renew만 기다리지 말고 fresh ticket을 발급한다.

```
sudo rm -f /run/user/<uid>/krb5cc
sudo systemctl start 'decs-krb-refresh@<username>.service'
sudo -u '#<uid>' klist -c FILE:/run/user/<uid>/krb5cc
```

4. 컨테이너 credential 확인

```
docker inspect <container> --format '{{range .Config.Env}}{{println .}}{{end}}' |  
grep -E '^(KRB5CCNAME|DECS_KRB5_PRINCIPAL|DECS_KERBEROS_ENABLED)='  
  
docker exec --user <username> <container> \  
sh -lc 'klist -c "$KRB5CCNAME" && touch ~/.__krb_test && rm ~/.__krb_test'
```

다음은 잘못된 상태다.

- container mount나 image 안에 *.keytab 이 있음
- KRB5CCNAME 이 host와 공유되지 않은 임시 경로를 가리킴
- container UID가 DB/AD UID와 다름
- Kerberos mode인데 unrestricted sudo와 광범위한 host mount를 함께 허용함

현재 저장소에는 Kubernetes Pod용 credential controller가 구현되어 있지 않다. Pod 운영을 추가한다면 Pod 생성 시 “어느 node에서 누가 keytab을 보관하고 ccache를 갱신할지”부터 구현해야 하며, Docker용 systemd timer 명령을 그대로 복사해 완료로 간주하면 안 된다.

5. NFS mount source 확인

환경마다 source, NFS version과 service principal이 다르다.

FARM의 올바른 형태는 다음과 같다.

```
nas.farm.decs.internal:/volume1/share /home/tako8/share nfs4  
defaults,vers=4.0,rsize=1048576,wsize=1048576,sec=krb5,proto=tcp,hard,timeo=600,retrans=2,addr=100.100.100.120,  
_netdev,exec,nouser 0 0
```

LAB의 올바른 형태는 다음과 같다. 실제 fstab에는 rpc-gssd 와 decs-kerberos-nfs-ready.service 에 대한 x-systemd.requires/after option도 playbook이 함께 추가한다.

```
lab-storage.lab.decs.internal:/294t/dcloud/share /home/tako9/share nfs4  
defaults,vers=4.1,sec=krb5,proto=tcp,hard,_netdev,exec,nouser 0 0
```

핵심은 source가 각 realm의 NFS service principal과 일치하는 FQDN이라는 점이다.

FARM 올바른: `nas.farm.decs.internal:/volume1/share`
FARM 잘못된: `100.100.100.120:/volume1/share` 또는 `192.168.2.30:/volume1/share`
FARM 정상: runtime option의 `addr=100.100.100.120`

LAB 올바른: `lab-storage.lab.decs.internal:/294t/dcloud/share`
LAB 잘못된: `100.100.100.100:/294t/dcloud/share` 또는 `192.168.1.20:/294t/dcloud/share`
LAB 정상: FQDN을 해석한 실제 transport 주소 `100.100.100.100`

IP source를 쓰면 `nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL` service principal 또는 `nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL` 과 이름이 맞지 않는다. `192.168.2.30` 과 `192.168.1.20` 은 관리 SSH 주소이며 운영 NFS data path가 아니다.

점검 명령:

```
MOUNT_TARGET=/home/tako9/share
STORAGE_FQDN=lab-storage.lab.decs.internal

findmnt -T "$MOUNT_TARGET" -o TARGET,SOURCE,FSTYPE,OPTIONS
nfsstat -m | sed -n "\\|${MOUNT_TARGET}|,+4p"
getent hosts "$STORAGE_FQDN"
```

FARM host에서는 값을 `/home/tako<번호>/share` 와 `nas.farm.decs.internal` 로 바꾼다. `findmnt` 의 `vers=4.0` 은 FARM, `vers=4.1` 은 LAB이어야 하고 두 환경 모두 `sec=krb5` 여야 한다.

`fstab` 설정은 [server-state playbook](#)에서 관리한다. 이미 mount된 legacy superblock은 유지보수 창에 `clean unmount/remount`한다. NFS caller가 D-state이면 강제 unmount를 반복하지 말고 포렌식 보존과 host reboot 판단으로 전환한다.

6. Machine credential과 NFS service ticket 확인

계산 호스트의 root mount에는 host machine keytab과 `rpc.gssd` 가 필요하다. realm과 NFS principal을 host 이름에 맞춰 선택한다.

```

short=$(hostname -s | tr '[:lower:]' '[:upper:]')

case "$short" in
    FARM*) KRB_REALM=FARM.DECS.INTERNAL
           KRB_NFS_PRINCIPAL=nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL ;;
    LAB*) KRB_REALM=LAB.DECS.INTERNAL
          KRB_NFS_PRINCIPAL=nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL ;;
    *)    printf '지원하지 않는 host: %s\n' "$short" >&2; exit 1 ;;
esac
KRB_MACHINE_PRINCIPAL="${short}\${KRB_REALM}"

sudo klist -kte /etc/krb5.keytab
sudo KRB5CCNAME=FILE:/run/decs-host-check.ccache \
    kinit -k -t /etc/krb5.keytab "$KRB_MACHINE_PRINCIPAL"
sudo KRB5CCNAME=FILE:/run/decs-host-check.ccache \
    kvno "$KRB_NFS_PRINCIPAL"
sudo rm -f /run/decs-host-check.ccache

systemctl status rpc-gssd --no-pager
systemctl cat rpc-gssd
pgrep -a rpc.gssd

```

`rpc.gssd`를 `-n`으로 시작하지 않는다. root mount는 host machine credential을 사용해야 하며, `-n`은 UID 0 사용자 credential을 찾게 하여 ticket이 없을 때 mount를 잃게 만들 수 있다.

7. Synology NAS와 Linux storage 운영 차이

FARM의 Synology NAS와 LAB의 Linux storage는 같은 NFS/Kerberos 개념을 사용하지만 설정을 관리하는 방식, service 이름과 keytab lifecycle이 다르다. client에서 확인하는 `findmnt`, `klist`, `kvno` 절차는 비슷해도 server 변경 명령은 서로 바뀌 실행하면 안 된다.

7.1 운영 경계 비교

항목	FARM Synology NAS	LAB Linux storage
관리 접속	<code>jy@192.168.2.30:6954</code>	<code>jy@192.168.1.20:6953</code>
AD join 도구	DSM UI 또는 <code>/usr/syno/sbin/synowin</code>	표준 Samba <code>net ads join</code>
Samba/winbind	Synology SMB package 경로와 <code>pkg-synosamba-*</code> unit	<code>/etc/samba/smb.conf</code> , 표준 <code>winbind.service</code>

항목	FARM Synology NAS	LAB Linux storage
RFC2307 NSS	<code>files winbind syno</code> 순서와 Synology idmap 설정을 함께 확인	<code>files systemd winbind</code> 와 Samba <code>idmap config</code> LAB : <code>backend = ad</code> 확인
NFS export root	<code>/volume1/share</code>	<code>/294t/dcloud/share</code> ; PoC는 별도 <code>test_krb</code> export
production export	storage IP별 <code>root_squash</code> , <code>sec=krb5:krb5i:krb5p</code> ; Synology <code>anonuid/anongid</code> , <code>insecure</code> option 포함	Linux <code>/etc/exports</code> 에서 <code>root_squash</code> , <code>sec=krb5</code> , <code>no_subtree_check</code> 를 명시
NFS service	Synology <code>/usr/sbin/svcgssd</code> 와 <code>/usr/sbin/idmapd</code>	<code>nfs-server</code> 와 <code>rpc-svcgssd</code> 또는 배포판의 socket-activated GSS service
NFS keytab	<code>/etc/nfs/krb5.keytab</code> 이 ticket 검증 기준; <code>/etc/krb5.keytab</code> 도 동기화 검사	<code>/etc/krb5.keytab</code> 한 경로
NFS SPN 등록 계정	전용 user <code>svc-nfs-farm</code> ; <code>NAS\$</code> 와 분리	computer account <code>LAB-STORAGE\$</code>
추가 acceptor	같은 keytab의 AILAB <code>nfs/nas.ai lab.dgu@AILAB.DGU</code> 를 반드시 보존	현재 없음
운영 NFS version	v4.0	v4.1
keytab checker	<code>--profile farm</code> , 5분 user timer 운영	<code>--profile lab</code> ; profile은 준비됐지만 PoC 단계에서는 timer를 기본 활성화하지 않음

Synology에서 `getent` , `id` 또는 `wbinfo` 가 DECS UID/GID 대신 `96470xxx` 같은 내부 ID를 반환하면 RFC2307 경로가 정상인 상태가 아니다. 이때 symbolic `chown -R 'FARM\user':'FARM\group'` 을 실행하면 잘못 해석된 숫자가 filesystem에 저장된다. 먼저 DB-AD와 NAS의 numeric UID/GID가 일치하는지 확인하고, ownership 복구도 검증된 숫자로 수행한다.

FARM NAS에는 관리·호환성 목적으로 남은 `192.168.2.0/24 sec=sys,no_root_squash` export가 있을 수 있다. 이것은 production Kerberos data path가 아니며 FARM host나 container가 이 `sec=sys` export를 사용하도록 변경하면 안 된다.

7.2 읽기 전용 점검

FARM Synology NAS에서는 vendor 경로와 service 이름을 사용한다.

```
ssh -p 6954 jy@192.168.2.30

/usr/syno/sbin/synowin -getWorkgroup
SMB=/usr/local/packages/@appstore/SMBService/usr/bin
sudo "$SMB/net" ads testjoin
sudo "$SMB/wbinfo" --online-status

sudo klist -kte /etc/nfs/krb5.keytab
sudo klist -kte /etc/krb5.keytab
sudo exportfs -v | sed -n '/\/volume1\/share/,+8p'
ps -ef | grep -E 'svcgssd|idmapd' | grep -v grep
```

LAB Linux storage에서는 표준 Samba/NFS systemd unit과 keytab을 확인한다.

```
ssh -p 6953 jy@192.168.1.20

sudo net ads testjoin
wbinfo --online-status
getent passwd '<username>'

sudo klist -kte /etc/krb5.keytab \
  | grep -F 'nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL'
systemctl is-active winbind nfs-server
systemctl --no-pager --full status rpc-svcgssd.service 2>/dev/null || true
sudo exportfs -v | sed -n '/\/294t\/dcloud\/share/,+8p'
```

LAB 배포판에서는 `rpc-svcgssd` 가 독립 unit이 아니라 socket 또는 `nfs-server` 의 일부로 관리될 수 있다. unit 이름 하나만으로 실패를 판정하지 말고 process, service keytab과 실제 `kvno` /canary 결과를 함께 본다.

7.3 변경 작업에서 지킬 차이

- Synology 설정을 바꾸기 전에는 DSM/Samba/NFS 설정과 두 keytab을 먼저 backup한다. DSM 또는 SMB package가 관리하는 파일·unit을 일반 Ubuntu와 같다고 가정하지 않는다.
- FARM keytab repair는 AILAB principal과 이전 FARM KVNO를 보존하는 전용 `repair-farm-nas-nfs-keytab.sh` 을 사용한다. `svcgssd` 를 FARM principal 하나로 제한하는 `-p` 나 nameless acceptor `-n` 으로 시작하지 않는다.
- LAB storage는 표준 Samba join, `/etc/krb5.keytab` , `nfs-server` / `rpc-svcgssd` 와 Ansible desired state를 기준으로 한다. FARM repair/rotation script를 LAB에 실행하지 않는다.
- 두 환경 모두 identity/export 변경 후 `exportfs -ra` 와 필요한 NFS/RPC cache flush를 수행할 수 있지만, 먼저 활성 client와 D-state를 확인하고 유지보수 시간에만 실행한다.
- Synology 관리 IP와 LAB storage 관리 IP는 NFS source가 아니다. 변경 후 client `findmnt` 에서 각각 `nas.farm.decs.internal` 과 `lab-storage.lab.decs.internal` 이 유지되는지 확인한다.

환경별 keytab 상태는 같은 checker를 profile만 바꿔 읽기 전용으로 확인한다.

```
cd /home/jy/server_manage/monitoring
health-checks/kerberos-nfs-keytab/script/check-nfs-keytab.sh --profile farm
health-checks/kerberos-nfs-keytab/script/check-nfs-keytab.sh --profile lab
```

server-side 전체 설정 절차는 [FARM canonical runbook](#)과 [LAB canonical runbook](#)을 각각 따른다.

8. FARM NAS service account와 KVNO 운영

현재 정책

- NFS SPN 등록 계정: `svc-nfs-farm`
- `NAS$`: domain membership용 `HOST/*`, `RestrictedKrbHost/*` 만 유지
- 자동 password expiration/rotation: 없음
- keytab drift 관측: 5분마다 읽기 전용
- repair/rotation: 관리자 명시 실행만 허용

과거 `NAS$` machine password의 주기적 변경으로 KVNO와 NAS NFS keytab이 어긋났기 때문에 전용 service account를 만들었다. 운영 문서에는 “KVNO가 주기적으로 바뀌어서 매번 새 계정을 만든다”가 아니라, **한 번 만든 전용 계정으로 자동 machine-account rotation에서 NFS SPN을 분리했다고** 기록한다.

읽기 전용 점검

```
cd /home/jy/server_manage/monitoring
health-checks/kerberos-nfs-keytab/script/check-nfs-keytab.sh --profile farm

systemctl --user status check-nfs-keytab@farm.timer --no-pager
systemctl --user list-timers 'check-nfs-keytab@farm.timer'
```

checker가 확인하는 항목:

- AD `svc-nfs-farm`의 현재 `msDS-KeyVersionNumber`
- NAS `/etc/krb5.keytab`, `/etc/nfs/krb5.keytab`에 현재 KVNO와 NFS principal 존재
- 공유 keytab에 AILAB acceptor가 보존되어 있는지
- 실제 `kvno -k` service-ticket 복호화 성공 여부
- `svcgssd` 실행 여부와 잘못된 `-p` 제한 여부

코드는 공용 checker, profile 값은 [FARM config](#)에서 확인한다.

Drift 수동 복구

```
cd /home/jy/server_manage/kerberos-nfs

./script/keytab/repair-farm-nas-nfs-keytab.sh --check
./script/keytab/repair-farm-nas-nfs-keytab.sh --repair
```

`--repair` 는 새 key를 export·검증하고 기존 AILAB principal과 이전 FARM KVNO를 보존하면서 NAS keytab을 원자 교체한다. 필요하다면 `svcgssd` 를 `-p` 없이 재시작하므로 활성 client 영향과 되돌릴 backup을 먼저 확인한다. 구현은 [repair script](#)에 있다.

계획된 rotation

```
./script/keytab/rotate-farm-nfs-service-key.sh --check

# 유지보수 창, AD replication, NAS/client 상태를 확인한 뒤에만 실행
./script/keytab/rotate-farm-nfs-service-key.sh --rotate --apply
```

rotation은 random password 설정, AD KVNO 변경, 새 key export, NAS keytab merge, `svcgssd` 반영, replica sync를 한 작업으로 수행한다. 이전 KVNO는 설정된 24시간 ticket lifetime보다 긴 최소 48시간 보존한다. 구현은 [rotation script](#)에 있다.

9. 모니터링에서 확인할 항목

무엇을 확인하는가	대표 상태/지표	코드
AD KVNO와 NAS keytab 일치	profile status JSON, checker exit 0/1/2	keytab health check
host keytab→kinit→kvno→rpc-gssd	<code>cluster_monitor_nfs_gss_ready</code> 와 stage	readiness script
실제 service path	<code>cluster_monitor_nfs_gss_canary_success</code>	canary probe
운영 mount	<code>cluster_monitor_host_mount_up</code> , <code>..._responsive</code> , probe seconds	cluster collector
missing mount 복구	<code>cluster_monitor_nfs_gss_recovery_*</code>	guarded recovery
D-state/lease/hung task	<code>cluster_monitor_host_dstate_*</code> , <code>cluster_monitor_nfs_*</code>	NFS forensics collector
경보 조건	<code>ClusterMonitorNFSGSS*</code> , <code>ClusterMonitorNFS*</code> , mount alerts	FARM Prometheus values

상세한 Prometheus/Grafana 운영은 [monitoring 운영 문서](#)를 참고한다.

10. 장애 진단 순서

사용자 한 명만 실패

1. DB UID/GID와 AD `uidNumber/gidNumber` 를 비교한다.
2. user keytab permission과 principal을 확인한다.
3. refresh service journal과 ccache `klist` 를 확인한다.
4. 동일 UID로 host NFS write를 시험한다.
5. container UID, `KRB5CCNAME` , ccache bind mount를 확인한다.
6. AD group 변경 직후라면 fresh ticket을 발급한다.

여러 host/user가 동시에 실패

1. `getent hosts <storage-fqdn>` 과 storage RPC 연결을 확인한다.
2. keytab checker에서 AD KVNO/NAS keytab drift를 확인한다.
3. `kvno nfs/<storage-fqdn>@<REALM>` 을 확인한다.
4. NAS `svcgssd` 와 service keytab을 확인한다.
5. mount source가 IP나 관리망 주소로 바뀌지 않았는지 확인한다.
6. AD replication 실패 시 실제 쓰기 대상 DC와 NAS가 조회하는 DC를 확인한다.

Mount가 없거나 응답하지 않음

1. `findmnt` 로 존재/source/security를 확인한다.
2. readiness 상태에서 처음 실패한 stage를 확인한다.
3. D-state, lease expiry, hung-task와 forensics snapshot을 확인한다.
4. mount가 **없는 경우에만** guarded recovery timer 상태를 확인한다.
5. 이미 healthy한 mount를 자동/수동으로 재마운트하지 않는다.
6. D-state caller가 남아 있으면 forced unmount 반복 대신 증거 보존 후 reboot를 검토한다.

11. 변경 전후 체크리스트

변경 전

- 대상 realm, DC, storage FQDN과 NFS principal 확인
- 활성 client와 유지보수 창 확인
- AD replication health 확인
- NAS keytab backup과 AILAB principal 확인
- 현재 `findmnt` , `klist` , checker status 보존

변경 후

- AD current KVNO와 NAS keytab KVNO 일치
- `kvno -k` 복호화 성공
- `svcgssd` 가 `-p` 없이 실행
- host readiness와 실제 사용자 NFS write 성공
- refresh timer enabled/active, ccache 유효
- Prometheus target/rules와 Grafana dashboard 정상
- 이전 KVNO를 최소 48시간 보존

12. 문서에 추가로 유지할 내용

설계와 운영이 다시 섞이지 않도록 다음 내용을 계속 분리해 기록한다.

- 설계 문서: trust boundary, principal naming, UID/GID invariant, ticket lifetime, FQDN 선택 근거, failure domain, Docker와 향후 Pod의 credential 모델
- 운영 문서: 현재 endpoint/KVNO가 아니라 확인 명령, timer/SLO, rotation 승인 조건, rollback, incident 진단 순서, 마지막 검증 날짜
- canonical runbook: 실제 host별 mount, 현재 적용 상태, 예외와 잔여 위험
- 실험 결과: 당시 kernel/NFS/security flavor와 재현 조건; 현재 default와 분리

keytab, password, local environment, incident 개인정보와 packet capture는 위키와 Git에 넣지 않는다.

Kerberos/NFS 디버깅 로그

이 문서는 Kerberos/NFS에서 발생한 장애와 재현 실험을 축적하는 인덱스다. 정상 상태를 만드는 절차는 [운영 문서](#)에 두고, 여기에는 장애가 어떻게 관측됐고 어떤 가설을 어떤 조건에서 검증했는지 기록한다.

기록 목록

검증일	상태	문제	핵심 결론
2026-07-20	관측, 인과 미확정	LAB9에서 <code>sec=krb5</code> 를 사용하는 이유	<code>krb5p</code> 에서 <code>krb5</code> 로 전환한 뒤 D-state 지속 시간, SUNRPC 적체와 mount probe 지연이 크게 감소함
2026-07-16	재현 완료, 과거 장애 원인 후보	NFSv4.1 session slot 고착	구형 storage kernel에서 지연된 idmap decode가 NFSv4.1 slot 하나를 영구 <code>INUSE</code> 상태로 남기는 경로를 재현함

상태는 다음 의미로 사용한다.

- **관측**: 증상과 증거는 있지만 원인 경로를 재현하지 못했다.
- **재현 완료**: 통제된 조건에서 같은 kernel 또는 protocol 경로를 다시 만들었다.

- **원인 후보:** 재현된 경로가 과거 장애를 설명하지만 장애 발생 시점의 직접 trace가 없어 동일 원인이었다고 확정할 수 없다.
- **원인 확정:** 장애가 시작된 시점의 증거와 재현 결과가 같은 경로를 가리킨다.

장애 발생 시 먼저 할 일

NFS **hard** mount 관련 process가 D-state에 들어간 경우에는 새로운 **find**, **du**, **stat**, canary I/O를 반복하지 않는다. 각 명령이 새로운 NFS request와 D-state process를 추가할 수 있다.

먼저 NFS 경로를 읽지 않는 local 상태만 수집한다.

```
date -Ins
uname -r
findmnt -rn -o TARGET,SOURCE,FSTYPE,OPTIONS

ps -e -o state=,pid=,ppid=,etimes=,comm=,wchan:48=,args= \
| awk '$1 ~ /^D/ {print}'

cat /proc/net/rpc/nfs 2>/dev/null || true
cat /run/decs-nfs-forensics/status.json 2>/dev/null || true
cat /var/lib/decs-nfs-gss/recovery.state 2>/dev/null || true
cat /var/lib/decs-nfs-gss/canary.state 2>/dev/null || true

systemctl --no-pager --full status rpc-gssd.service
journalctl -u rpc-gssd.service -b --no-pager -n 200
```

forensics가 배포된 host에서는 자동 snapshot이 생성됐는지 먼저 확인한다. 자동 trigger가 없었다면 local 상태만 읽는 수동 snapshot을 한 번 실행할 수 있다.

```
sudo systemctl start decs-nfs-forensics-snapshot.service
sudo cat /var/lib/decs-nfs-forensics/last_snapshot.state
```

이미 mount된 경로에 D-state caller가 있으면 **rpc-gssd** restart, forced unmount, 반복 remount를 먼저 실행하지 않는다. 증거를 보존한 뒤 storage-network 상태와 kernel wait path를 기준으로 복구 범위를 결정한다.

새 디버깅 문서 작성 형식

새 문제는 한 파일에 다음 내용을 남긴다.

1. **상태와 영향 범위:** 발생 시각, host, kernel, mount source/target/security
2. **증상:** 사용자 증상, process state, wait channel, alert
3. **확인한 사실과 가설:** 관측 사실과 아직 증명하지 못한 추론을 분리

4. **실험 목적**: 어떤 가설을 반증하거나 확인하려 했는지
5. **재현 조건**: server/client, version, cache, 동시성, fault와 안전장치
6. **실행 명령**: 사전 확인, fault 주입, workload, 관측, cleanup 순서
7. **결과와 판정 기준**: 성공/실패를 판단한 trace와 metric
8. **복구와 예방**: 임시 containment, 영구 수정, monitoring 변경
9. **원본 증거**: 결과 README, curated evidence, script와 checksum 링크

운영 환경에서 fault를 주입한 명령은 일반 runbook처럼 제시하지 않고 반드시 실행 일자와 승인·안전 조건을 붙인 **과거 실행 기록**으로 표시한다. 반복 실험은 가능하면 격리 VM harness로 옮긴다.

LAB9에서 **sec=krb5** 를 사용하는 이유

상태: LAB9에서 **sec=krb5p** 전환 전 24시간과 **sec=krb5** 전환 후 약 51시간을 비교한 운영 관측이다. 전환 뒤 NFS 지연과 적체가 크게 감소했지만, 관측 기간이 다르고 security flavor만 통제된 실험이 아니므로 **krb5p** 를 단일 원인으로 확정하지는 않는다.

1. 결론

LAB9의 Kerberos NFS 기본값은 **sec=krb5** 로 유지한다. 이 선택은 Kerberos principal 인증을 유지하면서 RPC payload 암호화 비용을 제외하고, LAB9에서 실제로 관측된 D-state 지속·SUNRPC 적체·mount probe 지연을 낮추기 위한 운영 기준이다.

krb5p 가 잘못된 방식이라는 뜻은 아니다. NFS payload 기밀성이 반드시 필요한 경로에서는 **krb5p** 를 사용해야 한다. 다만 현재 LAB9 사용자 공유의 기본 profile은 가용성과 지연 안정성을 우선해 **krb5** 를 사용한다. **krb5i** 또는 **krb5p** 는 요구사항이 있고 부하 검증을 통과한 경로에서만 명시적으로 선택한다.

2. security flavor의 차이

flavor	제공하는 보호	LAB9에서의 위치
sec=krb5	Kerberos principal 인증	기본값
sec=krb5i	인증 + RPC payload 무결성	변조 방지가 명시적으로 필요한 경로에서 검토
sec=krb5p	인증 + 무결성 + RPC payload 암호화	기밀성이 필요한 경로에서 성능 검증 후 사용

sec=krb5에서는 사용자와 NFS service가 Kerberos로 서로 인증되지만, NFS payload 자체는 암호화되지 않는다. 따라서 패킷 캡처와 forensics evidence는 root만 읽을 수 있게 보관하고, 보호되지 않은 네트워크에서 기밀 데이터를 전송해야 한다면 이번 성능 결과만으로 **krb5** 를 선택해서는 안 된다.

3. 왜 전환했나

`sec=krb5p` 사용 중 LAB9에서는 NFS 관련 D-state가 관측 기간 내내 존재했고, SUNRPC task와 transport reply 대기가 누적됐다. mount responsiveness probe도 설정된 10초 timeout 상한에 도달했다. 이 상태에서는 home 경로를 읽는 SSH나 사용자 process가 NFS 대기에 연쇄적으로 묶일 수 있다.

`krb5p` 는 각 RPC payload에 무결성 처리와 암호화를 추가한다. 이 비용이 장애의 단일 원인이라고 입증한 것은 아니지만, storage·kernel·network에서 이미 발생한 지연을 더 크게 보이게 하는 증폭 요인인지 확인할 운영상 이유가 있었다. 그래서 principal 인증은 유지하면서 privacy 처리를 제외하는 `sec=krb5` 로 전환하고 동일한 NFS health 지표를 계속 관측했다.

4. 전환 전후 결과

측정 대상은 LAB9다.

지표	<code>krb5p</code> 전환 전 24시간	<code>krb5</code> 전환 후 약 51시간
NFS D-state 존재 비율	100%	약 4.1%
D-state 프로세스 최대	28개	8개
최장 연속 D-state	83,339초(약 23.1시간)	1,236초(약 20분 36초)
<code>xprt_pending</code> 평균	5.35	1.29
RPC task 최대	3,696	525
마운트 probe 최대	10초	0.429초
lease 만료	없음	없음

변화 폭은 다음과 같다.

비교 항목	변화
NFS D-state 존재 비율	95.9%p 감소
D-state 프로세스 최대	약 71.4% 감소
최장 연속 D-state	약 98.5% 감소
<code>xprt_pending</code> 평균	약 75.9% 감소
RPC task 최대	약 85.8% 감소
마운트 probe 최대	약 95.7% 감소

가장 중요한 변화는 D-state가 “항상 존재하는 상태”에서 짧고 간헐적인 상태로 바뀐 점이다. `xprt_pending` 평균과 RPC task 최대도 함께 감소했으므로 단순히 D-state process 이름만 달라진 것이 아니라 NFS transport 적체 자체가 줄어든 방향과 일치한다.

전환 전 mount probe의 10초는 `cluster-monitor-exporter` 의 command timeout 상한과 같다. 따라서 이 값은 정상 응답에 10초가 걸렸다가보다 probe가 timeout 또는 기존 in-flight probe 상태에 도달했을 가능성이 크다. 전환 후 최대 0.429초는 관측 구간에서 10초 상한에 도달하지 않았음을 보여준다.

두 구간 모두 lease 만료가 없었다. 따라서 이번 비교에서 확인한 개선은 lease 만료 복구 여부보다 D-state 지속, RPC queue와 mount 응답성에 관한 것이다.

5. 지표의 의미

지표	판정 방법	구현
NFS D-state 존재 비율	유효 sample 중 NFS/RPC 관련 D-state process가 하나 이상인 sample 비율	<code>cluster_monitor_nfs_forensics_d_state_processes</code>
D-state 프로세스 최대	관측 구간의 NFS/RPC 관련 D-state process 최대값	같은 metric의 <code>max_over_time</code>
최장 연속 D-state	가장 오래 연속 관측된 NFS/RPC 관련 D-state process의 시간	<code>cluster_monitor_nfs_forensics_d_state_oldest_seconds</code>
<code>xprt_pending</code> 평균	transport reply를 기다리는 SUNRPC task 수의 구간 평균	<code>cluster_monitor_nfs_xprt_pending</code>
RPC task 최대	NFS RPC client task 수의 구간 최대	<code>cluster_monitor_nfs_rpc_tasks</code>
마운트 probe 최대	required mount에 수행한 bounded <code>statfs</code> 시간의 구간 최대	<code>cluster_monitor_host_mount_probe_seconds</code>
lease 만료	local NFSv4 lease deadline을 넘긴 시간	<code>cluster_monitor_nfs_lease_expired_seconds</code>

수집 경로는 다음과 같다.

```
LAB9 kernel:/proc 상태
-> NFS forensics watcher status.json
-> cluster-monitor-exporter metrics
-> FARM Prometheus
-> 같은 server="LAB9" label의 전환 전후 구간 집계
```

구현은 `forensics watcher`, `forensics metric adapter`, `mount/D-state collector`에서 확인한다.

6. 이 결과로 말할 수 있는 것과 없는 것

말할 수 있는 범위는 다음과 같다.

- LAB9에서 `krb5` 전환 뒤 모든 주요 NFS 적체 지표가 같은 방향으로 개선됐다.
- 약 51시간의 전환 후 구간에는 24시간 전환 전 구간과 같은 장시간 고착이 관측되지 않았다.

- Kerberos 인증을 유지한 `krb5` 가 LAB9의 현재 운영 baseline으로 더 안정적이었다.

아직 확정할 수 없는 범위는 다음과 같다.

- `krb5p` 의 암호화 비용만으로 전환 전 D-state가 발생했다는 인과관계
- 같은 시기에 달라졌을 수 있는 workload, storage queue, network, kernel 상태의 영향
- FARM/LAB의 다른 host에서도 감소 폭이 동일할 것이라는 일반화
- 51시간보다 긴 기간이나 peak workload에서도 같은 결과가 유지된다는 보장

따라서 이 기록의 판정은 **LAB9 운영 선택을 지지하는 강한 전후 관측**이며, 통제된 benchmark나 원인 확정 실험은 아니다. 원인을 더 좁히려면 동일 workload와 동일 시간대에 `krb5` / `krb5p` 를 반복 교차하는 격리 실험이 필요하다.

7. 운영 기준과 확인 방법

LAB9의 desired mount option은 `sec=krb5` 다. mount source는 IP가 아니라 NFS service principal과 일치하는 FQDN을 사용한다. D-state가 이미 존재할 때는 확인을 위해 mount를 반복해서 읽거나 즉시 remount하지 말고 먼저 local forensics 상태를 보존한다.

현재 mount의 security flavor는 다음처럼 정확히 확인한다.

```
LAB9_MOUNT_TARGET=/home/tako9/share

findmnt -T "$LAB9_MOUNT_TARGET" -n -o TARGET,SOURCE,FSTYPE,OPTIONS
findmnt -T "$LAB9_MOUNT_TARGET" -n -o OPTIONS \
| tr ' ' '\n' \
| grep -Fx 'sec=krb5'
```

`grep -F 'sec=krb5'` 만 사용하면 `sec=krb5p` 도 일치하므로 option을 쉼표 단위로 분리해 exact match한다. 전환 이후에도 최소한 다음 조건을 함께 확인한다.

- NFS D-state 존재 비율과 최장 연속 시간이 다시 증가하지 않는가
- `xprt_pending` 평균과 RPC task 최대가 전환 전 수준으로 돌아가지 않는가
- mount probe가 10초 timeout 상한에 다시 도달하지 않는가
- `cluster_monitor_nfs_lease_expired_seconds` 가 계속 0인가
- payload 기밀성 요구가 새로 생겨 `krb5p` 가 필요한 경로가 되지 않았는가

기본 security policy는 Kerberos/NFS 운영 저장소, 배포 값은 `monitoring exporter` 변수를 기준으로 한다. 상세 장애 채증과 D-state 대응은 [디버깅 로그 개요](#)의 절차를 따른다.

8. 다음 측정에서 함께 남길 값

현재 비교표에는 구간 길이만 있고 정확한 시작·종료 timestamp와 원본 query/export artifact가 없다. 다음 재측정부터는 아래 값을 함께 보관해야 같은 결과를 재계산할 수 있다.

- 전환 시각과 두 구간의 시작·종료 시각(타임존 포함)
- LAB9의 source FQDN, target, NFS version과 전체 mount option
- client/server kernel, storage 상태와 구간별 workload
- 사용한 PromQL 또는 `samples.tsv` 집계 script와 원본 결과
- exporter command timeout과 scrape interval
- 전환과 동시에 수행한 다른 설정 변경

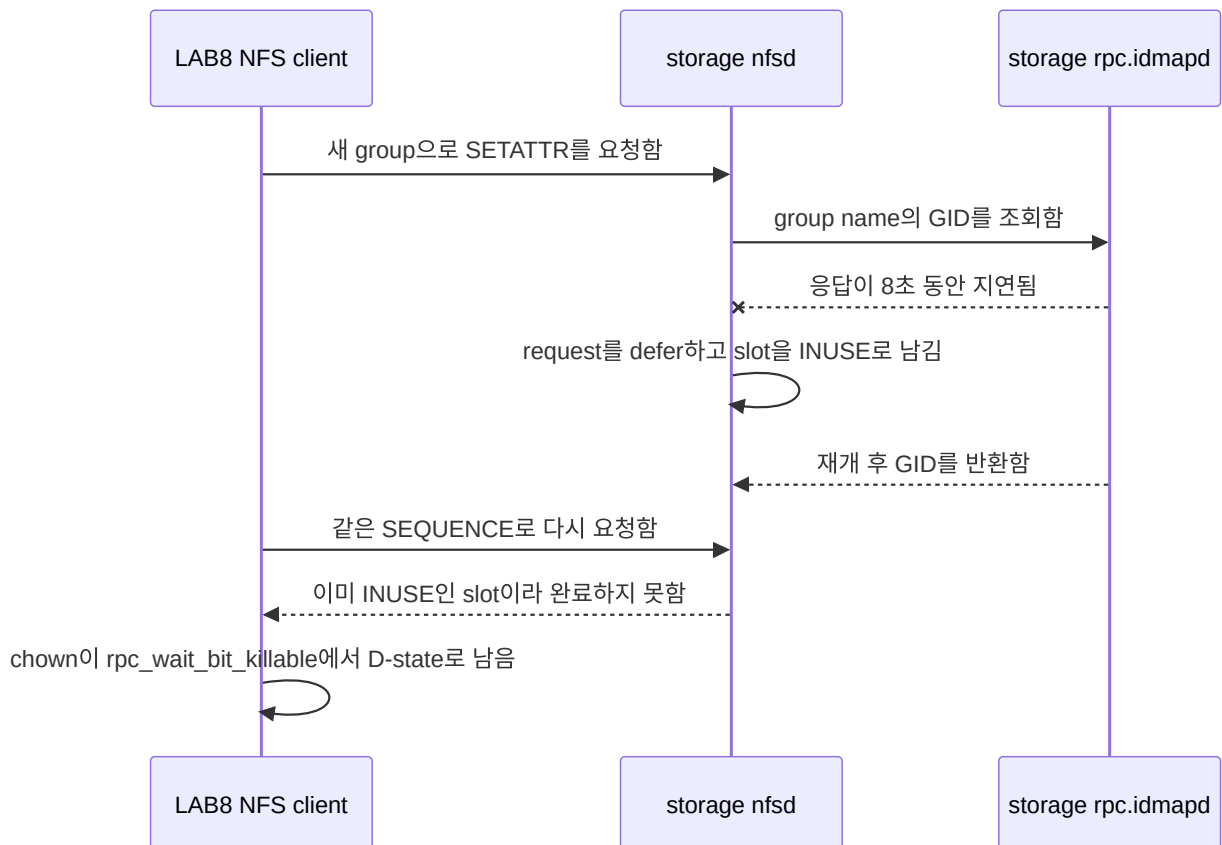
원본 artifact를 저장소에 추가하면 이 문서의 표 아래에 commit 고정 링크를 추가한다.

NFSv4.1 session slot 고착

상태: 2026-07-16에 slot leak 경로 재현 완료. 2026-06-29 LAB5 장애의 강한 원인 후보이지만, LAB5 장애 시작 시점의 storage trace가 없어 과거 장애의 최초 원인으로 확정하지는 않는다.

1. 요약

LAB storage의 구형 kernel `3.10.0-862.el7.x86_64` 에서 NFSv4 `SETATTR` 가 새로운 group 이름을 UID/GID로 해석하던 중 `rpc.idmapd` 응답이 지연되면 request가 defer될 수 있다. 이때 NFSv4.1 `SEQUENCE` 는 slot을 `INUSE` 로 표시하지만 response encode와 `nfsd4_sequence_done()` 이 실행되지 않아 slot이 반환되지 않는 경로를 실제로 재현했다.



한 slot만 leak한 최소 재현에서는 `ForeChannel` waiter가 0일 수도 있다. 따라서 waiter가 없다는 사실만으로 slot 문제를 배제하면 안 된다. session reset이나 recovery가 모든 slot의 drain을 기다리는 상황에서는 고착된 한 slot이 전체 복구를 막을 수 있다.

2. 어떤 증상이었나

2026-07-16 재현에서는 LAB8의 `chown` worker가 173초 이상 D-state에 남았다.

```
chown
-> nfs4_proc_setattr
-> nfs4_call_sync_sequence
-> rpc_wait_bit_killable
```

storage trace에서는 첫 처리와 재방문이 다음처럼 달랐다.

단계	관측
첫 idmap decode	<code>RQ_USEDEFERRAL</code> , <code>RQ_DROPME</code> 가 설정됨
첫 SEQUENCE 확인	<code>seqid=0x5f</code> , <code>slot_seqid=0x5e</code> , <code>INUSE=0</code>
첫 request 종료	dispatcher가 0을 반환했고 encode와 <code>sequence_done</code> 이 실행되지 않음
<code>rpc.idmapd</code> 재개 후	같은 request가 <code>seqid=0x5f</code> , <code>slot_seqid=0x5f</code> , <code>INUSE=1</code> 로 재방문
이후 retry	약 15초마다 같은 <code>INUSE=1</code> 을 확인하고 완료하지 못함

3. 왜 이 실험을 했나

조사는 다음 순서로 가설을 좁혔다.

- 2026-06-29 LAB5에서는 NFS user home을 읽는 `sshd: ... [priv]` 가 D-state에 누적됐다.
- 2026-07-10 LAB8에서 TCP/2049 transport를 일시 차단하자 같은 `rpc_wait_bit_killable` 계열 증상을 재현했지만, network가 복구된 뒤 process도 자연 복구됐다.
- keytab, `rpc-gssd`, readiness와 mount 순서를 바로잡은 clean boot에서도 구형 storage kernel의 session state가 영구 고착될 수 있는지는 별도 검증이 필요했다.
- upstream의 idmap deferral 수정 내용이 “NFSv4 decode 중 request drop으로 `NFSD4_SLOT_INUSE` 가 해제되지 않는다”는 경로를 설명하므로, 실제 LAB kernel과 mount에서 그 primitive를 최소 workload로 확인했다.

실험 목적은 많은 client를 동시에 멈추게 하는 것이 아니라, **idmap miss 하나가 slot 하나를 반환 불가능하게 만들 수 있는지**를 확인하는 것이었다.

4. 실험 결과

4.1 실제 LAB 환경의 최소 재현

첫 pass에서 `RQ_DROPME` 가 설정된 뒤 slot이 `INUSE` 로 바뀌었지만 encode와 `nfsd4_sequence_done()` 이 호출되지 않았다. 8초 뒤 `rpc.idmapd` 가 재개된 후에도 같은 sequence는 이미 사용 중인 slot으로 판단됐다. client retry도 동일 상태를 반복했고 worker는 D-state에서 끝나지 않았다.

따라서 다음 좁은 결론은 확정할 수 있다.

- 당시 storage kernel에서는 idmap decode deferral이 NFSv4.1 session slot 하나를 leak할 수 있다.
- keytab 발급, `rpc-gssd` 시작 순서나 client mount 순서만 고쳐서는 이 server-side kernel 경로를 제거할 수 없다.
- 한 slot leak은 재현했지만 LAB5에서 나중에 관측한 전체 session recovery/drain stall까지 이 최소 실험 하나로 재현한 것은 아니다.
- LAB5 장애 시작 시점의 trace가 없으므로 어떤 실제 operation이 최초 idmap miss를 만들었는지는 알 수 없다.

4.2 A-G: 8초 idmap 지연 kernel matrix

최소 재현과 같은 `idmap-setattr`, NFSv4.1, `sec=krb5p` 조건에서 server와 client kernel 조합을 바꿔 각 5회 실행했다. client OS는 모두 Ubuntu 22.04.5 LTS이며, F/G만 client kernel을 `6.18.38` 로 올렸다.

Case	NFS server OS	Server kernel	Client kernel	결과
A	CentOS Linux 7.9.2009	<code>3.10.0-862.el7.x86_64</code>	<code>6.8.0-134-generic</code>	slot 고착 5/5
B	CentOS Linux 7.9.2009	<code>3.10.0-1160.el7.x86_64</code>	<code>6.8.0-134-generic</code>	slot 고착 5/5
C	CentOS Stream 10	<code>6.12.74</code>	<code>6.8.0-134-generic</code>	slot 고착 5/5
D	CentOS Stream 10	<code>6.12.75</code>	<code>6.8.0-134-generic</code>	자동 복구 5/5
E	CentOS Stream 10	<code>6.12.0-248.el10.x86_64</code>	<code>6.8.0-134-generic</code>	자동 복구 5/5
F	CentOS Linux 7.9.2009	<code>3.10.0-862.el7.x86_64</code>	<code>6.18.38</code>	slot 고착 5/5
G	CentOS Linux 7.9.2009	<code>3.10.0-1160.el7.x86_64</code>	<code>6.18.38</code>	slot 고착 5/5

구형 server kernel에서는 client kernel을 `6.18.38` 로 올린 F/G도 모두 고착됐다. 반대로 idmap/slot fix가 포함된 D와 signed Stream kernel E는 모두 자동 복구했다. 이 결과는 문제와 복구 여부를 가르는 주된 변수가 client가 아니라 **NFS server kernel의 idmap deferral 처리**임을 지지한다.

4.3 D/E: 120·300·600초 장시간 idmap 지연 추가 실험

8초 지연만으로는 수정된 kernel이 짧은 지연에서만 우연히 복구한 것인지 판단하기 어려웠다. 그래서 자동 복구한 D/E만 대상으로 idmap cache miss 응답을 120초, 300초, 600초까지 지연하고 각 조건을 3회씩 다시 실행했다.

Case	Server kernel	idmap 지연	원래 chgrp	신규 RPC	최종 snapshot D / FC / lease
D-120s	6.12.75-nfs-slot	120초	성공 3/3	성공 3/3	모두 0 / 0 / 0*
D-300s	6.12.75-nfs-slot	300초	성공 3/3	성공 3/3	모두 0 / 0 / 0
D-600s	6.12.75-nfs-slot	600초	EINVAL 3/3	성공 3/3	모두 0 / 0 / 0
E-120s	6.12.0-248.el10.x86_64	120초	성공 3/3	성공 3/3	모두 0 / 0 / 0
E-300s	6.12.0-248.el10.x86_64	300초	성공 3/3	성공 3/3	모두 0 / 0 / 0
E-600s	6.12.0-248.el10.x86_64	600초	EINVAL 3/3	성공 3/3	모두 0 / 0 / 0

여기서 **FC** 는 NFSv4.1 ForeChannel waiter 수다. 600초 조건의 **EINVAL** 은 원래 **chgrp** 요청의 application 결과이며, slot 고착 판정과 분리해야 한다. 같은 run에서 idmap 지연을 해제한 뒤 실행한 신규 RPC는 모두 성공했고, 최종 snapshot의 D-state, ForeChannel waiter와 **lease_expired** 도 모두 0이었다. 따라서 600초에서 원래 작업이 실패했더라도 **session slot**은 반환됐고 후속 요청을 막지 않았다.

* D-120s 2회차의 2초 sampler는 종료 직전 D-state **1** 을 한 번 기록했다. 그러나 직후 final snapshot은 D-state **0** 이었고, 신규 RPC 성공, ForeChannel **0** , **lease_expired=0** , analyzer **verdict=pass** 였다. 지속된 NFS slot 고착으로 판정하지 않았다.

5. 실제 재현 조건

항목	2026-07-16 조건
client	LAB8, 100.100.100.108
server	lab-storage.lab.decs.internal , 100.100.100.100
mount	/home/tako8/share , NFSv4.1, sec=krb5p , hard
storage kernel	3.10.0-862.el7.x86_64
client 시작 상태	D-state 0, NFS RPC task 0, readiness ready
test identity	LAB8 local group decs_idmap_repro_424242 , GID 424242
test object	/home/tako8/share/.decs-idmap-slot-repro-20260716
fault	storage rpc.idmapd 에 8초간 SIGSTOP
workload	test file에 GID 424242 를 지정하는 chown(2) 한 번
안전장치	storage-local 8초·20초 SIGCONT timer를 먼저 예약
금지한 복구	재현 후 client reboot, unmount, signal, service restart를 하지 않음

boot 순서도 별도로 확인했다. LAB8은 `rpc-gssd` active → Kerberos/NFS readiness 완료 → mount 순서였고, 기존 `/etc/krb5.keytab`의 KVNO는 2였다. 따라서 이번 결과를 keytab 생성 누락이나 잘못된 mount 시작 순서로 설명할 수 없다.

6. 당시 실행한 명령

경고: 아래 명령은 2026-07-16의 승인된 실제 실행 기록이다. storage의 `rpc.idmapd`를 멈추므로 일반 점검이나 운영 재현 절차로 다시 실행하면 안 된다. 재실험은 8절의 격리 VM harness를 사용한다. 특히 `kprobe` 인자 register는 당시 x86_64 kernel symbol에만 맞춘 값이라 다른 kernel에서 그대로 사용하지 않는다.

6.1 client 사전 확인과 test 값 준비

LAB8에서 D-state와 RPC task가 없는지 확인한 뒤, storage에 없던 group 이름을 client가 encode하도록 local group과 test file을 준비했다.

```
ps -eo stat= | awk '$1 ~ /^D/ {n++} END {print "d_state=" (n+0)}'
grep -c '^RPC task' /proc/net/rpc/nfs 2>/dev/null || true
findmnt -T /home/tako8/share -n -o TARGET,SOURCE,FSTYPE,OPTIONS

sudo groupadd -g 424242 decs_idmap_repro_424242

sudo bash -c '
p=/home/tako8/share/.decs-idmap-slot-repro-20260716
rm -f "$p"
touch "$p"
chmod 0666 "$p"
stat -c "path=%n uid=%u gid=%g mode=%a ino=%i" "$p"
'
```

fault 전에 GID `424241`을 사용한 정상 `chown`을 실행해 같은 경로가 idmap 지연 없이는 encode와 `sequence_done`까지 완료되는 것을 먼저 확인했다.

```
sudo python3 -c 'import os; p="/home/tako8/share/.decs-idmap-slot-repro-20260716"; print("before",
os.stat(p).st_gid); os.chown(p, -1, 424241); print("after", os.stat(p).st_gid)'
```

6.2 storage trace 설정

당시 storage에서 idmap decode, slot 검사, dispatcher return, response encode와 slot 반환 여부를 한 trace instance에 기록했다.


```

T=/sys/kernel/debug/tracing
I="$T/instances/decs-idmap-slot-repro"
sudo mkdir -p "$I"

for event in uid gid check_slot dispatch dispatch_ret encode sequence sequence_done; do
    echo "-:decs_idmap/$event" | sudo tee -a "$T/kprobe_events" >/dev/null || true
done

echo 'p:decs_idmap/uid nfsd_map_name_to_uid rqstp=%di name=+0(%si):string namelen=%dx:u32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/gid nfsd_map_name_to_gid rqstp=%di name=+0(%si):string namelen=%dx:u32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/check_slot check_slot_seqid seqid=%di:u32 slot_seqid=%si:u32 slot_inuse=%dx:u32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/dispatch nfsd_dispatch rqstp=%di' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'r:decs_idmap/dispatch_ret nfsd_dispatch ret=$retval:s32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/encode nfs4svc_encode_compoundres rqstp=%di' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/sequence nfsd4_sequence rqstp=%di cstate=%si seq=%dx' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/sequence_done nfsd4_sequence_done resp=%di' \
    | sudo tee -a "$T/kprobe_events" >/dev/null

echo 0 | sudo tee "$I/tracing_on" >/dev/null
echo 0 | sudo tee "$I/events/enable" >/dev/null
sudo sh -c ": > '$I/trace'"
echo 8192 | sudo tee "$I/buffer_size_kb" >/dev/null

for event in \
    decs_idmap/uid decs_idmap/gid decs_idmap/check_slot decs_idmap/dispatch \
    decs_idmap/dispatch_ret decs_idmap/encode decs_idmap/sequence \
    decs_idmap/sequence_done sunrpc/svc_process sunrpc/svc_defer \
    sunrpc/svc_revisit_deferred sunrpc/svc_drop sunrpc/svc_send
do
    echo 1 | sudo tee "$I/events/$event/enable" >/dev/null
done
echo 1 | sudo tee "$I/tracing_on" >/dev/null

```

6.3 fault와 단일 workload 실행

storage에서 반드시 자동 재개 timer 두 개가 active인지 확인한 뒤 `rpc.idmapd`를 멈췄다.

```
pid="$(pgrep -xo rpc.idmapd)"

sudo systemd-run --unit=decs-idmapd-auto-resume-8s \
  --on-active=8s --timer-property=AccuracySec=1s \
  /bin/kill -CONT "$pid"
sudo systemd-run --unit=decs-idmapd-auto-resume-20s \
  --on-active=20s --timer-property=AccuracySec=1s \
  /bin/kill -CONT "$pid"

sudo systemctl is-active --quiet decs-idmapd-auto-resume-8s.timer
sudo systemctl is-active --quiet decs-idmapd-auto-resume-20s.timer
sudo kill -STOP "$pid"
```

그 직후 LAB8의 별도 systemd worker에서 `chown(2)` 한 번만 실행했다.

```
sudo systemd-run --unit=decs-idmap-slot-repro-worker --no-block \
  /usr/bin/python3 -c \
  "import os; os.chown('/home/tako8/share/.decs-idmap-slot-repro-20260716', -1, 424242)"
```

6.4 결과 확인과 trace 보존

새로운 NFS I/O를 만들지 않고 worker, local kernel state와 이미 동작 중이던 forensics state를 확인했다.

```
systemctl --no-pager --full status decs-idmap-slot-repro-worker.service
ps -eo pid=,ppid=,stat=,comm=,wchan:32=,args= \
  | awk '$3 ~ /^D/ {print}'
grep -E 'lease_expired|RPC task|xprt_pending|ForeChannel' \
  /proc/net/rpc/nfs 2>/dev/null || true

cat /run/decs-nfs-forensics/status.json
cat /var/lib/decs-nfs-gss/recovery.state
cat /var/lib/decs-nfs-gss/canary.state
```

storage trace는 다음 pattern을 기준으로 추렸다.

```
I=/sys/kernel/debug/tracing/instances/decs-idmap-slot-repro
sudo sh -c "echo 0 > '$I/tracing_on'"
sudo cp "$I/trace" /var/tmp/decs-idmap-slot-repro-20260716.trace

sudo grep -E \
  'decs_idmap_gid|svc_defer|svc_revisit_deferred|svc_drop:|check_slot|sequence_done|dispatch_ret' \
  /var/tmp/decs-idmap-slot-repro-20260716.trace
```

7. Monitoring과 운영 대응

재현 당시 guardrail은 의도대로 동작했다.

- forensics watcher가 `nfs_d_state` 이유로 incident snapshot을 보존했다.
- recovery는 `status=blocked`, `diagnosis=nfs_d_state_detected`, `retry_inhibited=1`로 바뀌었다.
- canary도 같은 진단으로 차단돼 추가 NFS request를 만들지 않았다.
- packet ring과 kernel trace는 계속 증거를 수집했다.

운영에서 같은 증상이 보이면 다음 순서를 따른다.

1. NFS path를 읽는 `find`, `du`, `stat`, canary를 추가 실행하지 않는다.
2. D-state stack, `/proc/net/rpc/nfs`, forensics status와 snapshot을 보존한다.
3. mount가 이미 있으면 자동 remount와 `rpc-gssd` 반복 restart를 차단한다.
4. storage kernel, `rpc.idmapd`, network와 NFS session 증거를 함께 확인한다.
5. 상태가 자연 복구되지 않으면 유지보수 창외 client reboot 또는 storage 조치를 수동 결정한다. reboot는 상태를 지울 뿐 원인 수정은 아니다.
6. 영구 조치는 idmap deferral fix가 포함된 storage kernel로 올리고 격리 회귀 실험을 통과시키는 것이다.

운영 mount를 `soft`로 바꾸는 방식은 timeout을 application error로 노출하고 data integrity 위험을 만들 수 있어 해결책으로 사용하지 않는다.

8. 격리 VM에서 다시 검증하는 방법

반복 실험은 LAB8 local disk의 `decs-nfs-slot-isolated` network에서 수행한다. 이 network는 forwarding이 없고 `100.100.100.0/24`, `192.168.1.0/24`와 운영 storage hostname을 거부한다. VM disk, trace와 result도 NFS share가 아니라 LAB8 local filesystem에 둔다.

kernel 비교 기준은 다음과 같다.

Case	Server kernel	Client kernel	목적
A	CentOS 7 <code>3.10.0-862</code>	Ubuntu <code>6.8.0-134</code>	운영 취약 baseline
B	CentOS 7 <code>3.10.0-1160</code>	Ubuntu <code>6.8.0-134</code>	CentOS 7 최종 kernel 비교

Case	Server kernel	Client kernel	목적
C	upstream 6.12.74	Ubuntu 6.8.0-134	fix 이전 positive control
D	upstream 6.12.75	Ubuntu 6.8.0-134	idmap/slot fix 비교 control
E	signed Stream 6.12.0-248.el10	Ubuntu 6.8.0-134	운영 후보
F	CentOS 7 3.10.0-862	kernel.org 6.18.38	새 client kernel 비교
G	CentOS 7 3.10.0-1160	kernel.org 6.18.38	새 client kernel 비교

준비와 격리는 VM lab README의 절차를 따른다.

```
cd /home/jy/server_manage/kerberos-nfs/test/lab-kerberos-poc/vm-slot-regression

sudo ./bin/provision_lab8_vms.sh "$PWD"
./bin/configure_vm_stack.sh
sudo ./bin/prefetch_kernel_matrix.sh
sudo ./bin/verify_kernel_sources.sh
sudo ./bin/build_upstream_kernels_vm.sh
sudo ./bin/prepare_centos7_domains.sh "$PWD"
sudo ./bin/isolate_test_vms.sh
```

단일 비교는 같은 idmap-setattr, sec=krb5p, 8초 조건을 case별로 실행한다.

```
./bin/switch_kernel_case.sh A
NFS_SLOT_EXPECT_VULNERABLE=1 \
./bin/run_single_case.sh A krb5p idmap-setattr 8 1

./bin/switch_kernel_case.sh D
NFS_SLOT_EXPECT_VULNERABLE=0 \
./bin/run_single_case.sh D krb5p idmap-setattr 8 1
```

먼저 실행 수만 확인하려면 plan-only mode를 쓴다.

```
MATRIX_PLAN_ONLY=1 \
MATRIX_CASES='A C D E' \
MATRIX_SECURITY='krb5p' \
MATRIX_SCENARIOS='idmap-setattr' \
MATRIX_DURATIONS='8' \
MATRIX_REPETITIONS=3 \
./bin/run_matrix.sh
```

취약 case가 고착되면 harness는 evidence를 archive한 뒤 **격리 VM만** reset한다. 그 reset은 자동 복구 성공으로 계산하지 않는다. 운영 LAB8 host와 운영 NFS mount는 이 harness의 복구 대상이 아니다.

9. 원본 증거와 코드

- 2026-07-16 실제 재현 결과
- curated trace evidence
- 2026-07-10 NFS transport fault 실험
- 격리 VM slot regression lab
- single-case harness
- result analyzer
- upstream stable patch 설명
- NFS forensics 구현

보존된 storage raw trace의 SHA-256은 `5fe7fbab2b8aa712ff71afe5156438b4e880edcc39cf16a00592e22d578bac59` 다.